

به نام خدا



درس سیستم عامل
نیم سال اول ۰۵-۰۴
استاد: دکتر اسدی

دانشکده مهندسی کامپیوتر

تمرین سری دوم

- پرسش‌های خود را در سامانه CW و تالار مربوط به تمرین مطرح نمایید.
- پاسخ سوالات را تایپ نمایید.
- اسکرین‌شات‌ها، عکس‌ها، فایل‌های مربوط به سوال عملی، گزارش تمرینات عملی و PDF قسمت تئوری را در پوشه با نام به فرمت HWNUM_StudentID1_StudentID2 ذخیره نمایید. سپس آن را zip نمایید و در صفحه درس بارگذاری نمایید. به عنوان مثال یک فایل بارگذاری شده قابل قبول باید دارای فرمت HW1_400123456_403123456.zip باشد.
- هر دانشجو می‌تواند حداکثر سه تمرین را با دو روز تأخیر بدون کاهش نمره ارسال نماید.
- تمرینات عملی و تئوری به صورت گروه‌های دو نفر تحویل داده شود.
- هر دو عضو گروه موظف هستند تمرینات خود را بارگذاری کنند.
- عواقب عدم تطابق بین پاسخ دو عضو گروه برعهده خودشان است.
- تحویل تمرین به صورت انگلیسی مجاز نیست. در صورت تحویل تمرین به صورت انگلیسی (حتی بخشی از تمرین) نمره تمرین موردنظر صفر در نظر گرفته می‌شود.
- در صورت مشاهده تقلب برای بار اول نمره هر دو طرف صفر می‌شود. در صورت تکرار نمره کل تمرینات صفر خواهد شد.
- استفاده از ابزارهایی مانند ChatGPT به منظور ابزار کمک آموزشی مجاز است به شرط آن که به خروجی آن اکتفا نشود.
- توجه شود که پروژه نهایی درس در گروه‌های چهار نفر تحویل گرفته می‌شود.
- سوالات با عنوان اختیاری نمره‌ای ندارند اما جواب دادن به آن‌ها کمک به سزایی در یادگیری درس می‌کند.
- هیچ تمرینی خارج از CW تصحیح نخواهد شد. لذا توصیه می‌شود حداقل یک الی دو ساعت قبل از زمان پایان تمرین برای بارگذاری تمرین اقدام شود.

تمارین تئوری

۱. طراحی سیستم عامل به صورت لایه‌ای باعث ساده‌تر شدن ساختار و تفکیک وظایف می‌شود، اما لایه‌بندی بیش از حد می‌تواند کارایی را کاهش دهد و نگهداری سیستم را پیچیده کند.

(آ) توضیح دهید که چگونه لایه‌بندی سخت‌گیرانه می‌تواند اشکال‌زدایی، گسترش‌پذیری^۱ و کارایی در زمان اجرا را هم بهبود دهد و هم مانع آن‌ها شود. مثال‌های واقعی بیاورید (مثل پشته‌ی I/O^۲ در لینوکس یا لایه‌ی VFS^۳).

(ب) تحلیل کنید که هسته‌های مبتنی بر ماژول (مثل ماژول‌های قابل بارگذاری لینوکس) چگونه با طراحی‌های صرفاً لایه‌ای تفاوت دارند و در دستیابی به انعطاف‌پذیری زمان اجرا و ایزوله کردن چه مزایایی دارند.

(ج) توضیح دهید که چرا در برخی عملیات حساس از نظر کارایی (مثل پردازش بسته‌های شبکه یا زمان‌بندی دیسک) قوانین لایه‌بندی نادیده گرفته می‌شود. این کار چه خطراتی ایجاد می‌کند؟

۲. به سوالات زیر پاسخ دهید:

(آ) فرض کنید که شما قرار است که یک بازی کامپیوتری که برای سیستم عامل ویندوز پردازنده‌های x86-64 نوشته شده است را بر روی سیستم عامل لینوکس بر روی معماری x86-64 اجرا کنید. آیا برای این کار حتماً نیاز به یک ماشین مجازی ویندوزی دارید؟ اگر حتماً نیاز به یک ماشین مجازی دارید دلیل بیاورید و در غیر این صورت راه حل خود را برای اجرای این بازی روی لینوکس را شرح دهید.

(ب) حال فرض کنید که می‌خواهید یک برنامه که برای سیستم عامل لینوکس ARM ۶۴ بیتی کامپایل شده است را روی سیستم عامل لینوکسی که روی معماری x86-64 اجرا می‌شود، اجرا کنید. در ابتدا توضیح دهید که چرا به کمک مجازی‌سازی نمی‌توان این برنامه را اجرا کرد. آیا برای اجرای این برنامه نیاز به شبیه‌سازی کل هسته‌ی سیستم عامل لینوکس ARM ۶۴ بیتی داریم؟ در صورتی که جواب شما بله است دلیل بیاورید و در غیر این صورت راه حل خود را برای اجرای این برنامه بدون شبیه‌سازی هسته ارائه دهید.

(ج) حال به کمک دو سوال بالا بررسی کنید که چگونه می‌توان برنامه‌هایی که برای سیستم عامل ویندوز معماری x86-64 نوشته شده‌اند را روی سیستم عامل لینوکسی که روی یک پردازنده‌ی ARM ۶۴ بیتی اجرا می‌شود، اجرا کرد.

(د) (اختیاری) تحقیق کنید که استاندارد POSIX چیست و چه کمکی به اجرای یک برنامه روی سیستم عامل‌های مختلف می‌کند.

(ه) (اختیاری) سیستم عامل ویندوز هیچ مستنداتی درباره‌ی لیست فراخوانی‌های سیستمی خود به کاربران ارائه نمی‌دهد و اگر در برنامه‌ای از فراخوانی سیستمی استفاده شود، به احتمال بالا توسط آنتی ویروس سیستم عامل به عنوان بدافزار تشخیص داده می‌شود. در ابتدا تحقیق کنید که چه روش جایگزینی برای استفاده از فراخوانی‌های سیستمی برای برنامه‌های ویندوزی موجود است و سپس توضیح دهید که دلیل پنهان کردن فراخوانی‌های سیستمی چه چیزی می‌تواند باشد.

۳. در برخی از سیستم عامل‌ها، یک رابط واحد برای فراخوانی‌های سیستمی^۴ در نظر گرفته می‌شود که هم برای فایل‌ها و هم برای دستگاه‌ها مورد استفاده قرار می‌گیرد (برای مثال، توابعی مانند `open()`، `read()`، `write()`، `close()` برای هر دو مورد به کار می‌روند).

(آ) مزایای معماری این یکپارچگی را از نظر انتزاع‌سازی^۵ و قابلیت گسترش‌پذیری^۶ توضیح دهید.

(ب) فراخوانی سیستمی `ioctl` چه کار می‌کند و چه محدودیتی را رفع می‌کند؟

^۱Extensibility

^۲Input/Output

^۳Virtual File System

^۴System Calls (Syscall)

^۵Abstraction

^۶Extensibility

(ج) در یک سیستم میکرو هسته^۷، این فراخوانی‌ها معمولاً از طریق پیام‌رسانی^۸ به سرورهای فایل و دستگاه در فضای کاربر انجام می‌شوند. تجزیه و تحلیل کنید که این طراحی چگونه بر کارایی^۹، قابلیت اطمینان^{۱۰} و امنیت^{۱۱} در مقایسه با یک هسته یک پارچه^{۱۲} که از همان رابط استفاده می‌کند تأثیر می‌گذارد.

(د) یک روش بهینه‌سازی پیشنهاد دهید که بتواند سربار^{۱۳} ناشی از این فراخوانی‌های سیستمی مبتنی بر پیام را کاهش دهد، در حالی که همچنان جداسازی^{۱۴} در معماری میکرو هسته حفظ شود.

۴. فرض کنید شما مسئول طراحی سیستم عامل برای سه محیط کاملاً متفاوت هستید:

• سیستم A: یک ماهواره فضایی که باید ۱۰ سال بدون نیاز به به روز رسانی کار کند

• سیستم B: یک سرور ابری که میزبان هزاران ماشین مجازی است

• سیستم C: یک ساعت هوشمند با باتری محدود و حافظه ۵۱۲ مگابایت

(آ) برای هر سیستم، کدام ساختار (یکپارچه^{۱۵}، ریزهسته^{۱۶}، لایه‌ای^{۱۷}، یا ماژولار^{۱۸}) را انتخاب می‌کنید. چرا؟

(ب) کدام خدمات سیستم عامل (از جمله مدیریت فرآیند^{۱۹}، مدیریت حافظه^{۲۰}، سامانه پرونده^{۲۱}) در هر سیستم اولویت دارند و کدام‌ها می‌توانند حذف یا ساده‌سازی شوند؟

(ج) چگونه استراتژی مدیریت وقفه‌ها و فراخوانی‌های سیستمی در هر یک از این سیستم‌ها متفاوت خواهد بود؟

۵. به سوالات زیر درباره فراخوانی‌های سیستمی پاسخ دهید.

(آ) هنگام استفاده از فراخوانی‌های سیستمی رویه‌ی متداول این است که در ابتدا آرگومان‌های پاس داده شده از پشته‌ی^{۲۲} کاربر به پشته‌ی هسته کپی می‌شوند و سپس صحت آرگومان‌ها چک می‌شود. چرا ترتیب این دو مرحله نمی‌تواند برعکس باشد؟ به عبارت دیگر چرا نمی‌توان در ابتدا صحت آرگومان‌ها را بررسی کرد و سپس به پشته‌ی هسته کپی شوند؟ در این سوال منظور از صحت آرگومان به عنوان مثال بررسی بازه‌های اعداد یا فرمت یک رشته‌ی پاس داده شده است.

(ب) آیا راهی به غیر از استفاده از فراخوانی‌های سیستمی وجود دارد که کاربر با دسترسی سطح کاربر کدی بنویسد و اجرا کند که سیستم را وادار به انتقال به حالت هسته کند؟ اگر ندارد چرا و اگر دارد مثال بزنید.

(ج) برای اجرای یک فراخوانی سیستمی معمولاً از یک جدول استفاده می‌شود. به این صورت که در سطح کاربر شماره‌ی متناظر با فراخوانی سیستمی مد نظر وارد یک ثبات^{۲۳} می‌شود و از آن انتقال به سطح هسته با بررسی شماره‌ی فراخوانی سیستمی، آدرس دستورالعمل‌هایی که باید اجرا شوند پیدا می‌شود. چرا به جای این کار در سطح کاربر مستقیماً آدرس توابع هسته را در ثبات‌ها قرار نمی‌دهیم؟ حداقل دو دلیل بنویسید.

⁷Microkernel

⁸Message Passing

⁹Performance

¹⁰Reliability

¹¹Security

¹²Monolithic Kernel

¹³Overhead

¹⁴Isolation

¹⁵Monolithic

¹⁶Microkernel

¹⁷Layered

¹⁸Modular

¹⁹Process Management

²⁰Memory Management

²¹File System

²²Stack

²³Register

۶. امروزه به واسطه‌ی گسترش اینترنت و پردازش لبه^{۲۴} بازار سرورهای مجازی شخصی^{۲۵} بسیار داغ شده است. سرورهای مجازی، سیستم‌های مجازی‌سازی شده‌ای هستند که روی یک سرور اختصاصی^{۲۶} در حال اجرا هستند و هر یک از آن‌ها به صورت تام در اختیار یک مشتری قرار می‌گیرد. یکی از معضله‌هایی که شرکت‌های ارائه دهنده‌ی این خدمات با آن مواجه هستند، انتخاب Hypervisor مورد استفاده برای انجام عملیات مجازی‌سازی است. به عنوان مثال امروزه برنامه‌های مجازی‌سازی مانند ESXi و KVM و Hyper-V بسیار معروف و مورد استفاده هستند. با توجه به این مقدمه، درباره‌ی سوالات زیر تحقیق کنید:

- (آ) بهتر است که برای راه اندازی سرورهای مجازی از مجازی‌سازی نوع اول یا دوم استفاده کنیم؟ چرا؟
- (ب) مجازی‌سازی‌های ESXi و KVM و Hyper-V را مقایسه کنید و برای هر یک، یک برتری و عیب آن نسبت به بقیه‌ی مجازی‌سازها را نام ببرید.
- (ج) تحقیق کنید OpenVZ و LXC چیست و چگونه کار می‌کنند. تفاوت اصلی این دو برنامه نسبت به مجازی‌سازهای نوع اول چیست؟
- (د) فرض کنید می‌خواهیم یکی از دو برنامه LXC و KVM را به عنوان مجازی‌ساز انتخاب کنیم. یکی از فاکتورهای موثر در تصمیم‌گیری و تاثیر آن‌ها بر انتخابمان برای هر یک از این برنامه‌ها را بنویسید.
- (ه) (اختیاری) OpenVZ و LXC را با هم مقایسه کنید. هر کدام چه ویژگی نسبت به دیگری دارند؟
- (و) (اختیاری) تحقیق کنید که WSL^{۲۷} بر روی سیستم‌عامل ویندوز از چه روشی برای اجرای برنامه‌های لینوکسی مختلف استفاده می‌کند. WSL1 و WSL2 چه فرقی با هم دارند؟

۷. (اختیاری) به واسطه‌ی محبوب شدن هوش مصنوعی، تقاضای استفاده از کارت‌های گرافیک در سرورهای مجازی افزایش یافته است. از همین جهت عرضه و تقاضای کارت‌های گرافیکی در دیتاسنترها با مشکل مواجه شده است. یکی از روش‌هایی که می‌توان کمبود عرضه را جبران کرد مجازی‌سازی کارت‌های گرافیک و به اشتراک گذاری منابع آن بین چندین ماشین مجازی است. در رابطه با این موضوع به سوالات زیر فکر کنید. (دقت کنید که تمامی زیر بخش‌ها اختیاری هستند)

- (آ) چه چالش‌هایی در مجازی‌سازی کارت‌های گرافیکی وجود دارد؟
- (ب) SR-IOV یا Single-root Input Output Virtualization چیست و چه کمکی به مجازی‌سازی کارت‌های گرافیکی می‌کند؟
- (ج) vGPU در کارت‌های گرافیکی NVIDIA چیست؟ Multi-Instance GPU چه تفاوتی با آن دارد؟
- (د) Virtio Venus چیست؟ استفاده از Venus چه محدودیت‌هایی را می‌تواند به همراه داشته باشد؟
- (ه) تحقیق کنید که Hyper-V چگونه مشکل مجازی‌سازی کارت‌های گرافیکی را حل می‌کند و چه محدودیت‌هایی را به همراه دارد؟
- (و) سناریویی را مثال بزنید که در آن حتماً به مجازی‌سازی کارت‌های گرافیک نیاز داریم و سناریو دیگری را مثال بزنید که صرفاً با یک کانیتینر داکر و دادن کارت گرافیک به داکر می‌توان خواسته‌ی مشتری را برطرف کرد.

²⁴Edge Computing

²⁵Virtual Private Servers (VPS)

²⁶Dedicated Server

²⁷Windows Subsystem for Linux

تمارین عملی

۱. در سوال عملی مربوط به xv6 این سری از تمارین می‌خواهیم که هم با GDB برای دیباگ کردن سیستم‌عامل xv6 آشنا شویم و هم اینکه یک فراخوانی سیستمی به سیستم‌عامل اضافه کنیم که برخی از اطلاعات مربوط به سیستم‌عامل را بر می‌گرداند. برای هر کدام از قسمت‌های این سوال یک فیلم آموزشی در cw قرار داده شده است که می‌تواند به شما کمک کند. در ادامه هم پیشنهاد می‌کنیم که بخش‌های ۴.۳ و ۴.۴ کتاب سیستم‌عامل xv6 را مطالعه کنید.

آ- در بسیاری از موارد استفاده از دستورات چاپ برای دیباگ کردن هسته کافی است، اما در بعضی موارد امکان حرکت کردن بین دستورات اسمبلی یا بررسی متغیرهای درون پشته می‌تواند فرایند دیباگ را برای ما آسان‌تر کند.

برای اینکه بیشتر با GDB آشنا شوید، ابتدا دو پنجره باز کنید. در پنجره اول دستور `make qemu-gdb` را اجرا کنید و gdb را در پنجره دیگر باز کنید. زمانی که دو پنجره را باز کرده‌اید، دستور مقابل را در پنجره اجرا کنید:

```

1 (gdb) b syscall
2 Breakpoint 1 at 0x80002142: file
3 kernel/syscall.c, line 243.
4 (gdb) c Continuing.
5 [Switching to Thread 1.2]
6
7 Thread 2 hit Breakpoint 1, syscall () at
8 kernel/syscall.c:243
9 243 {
10 (gdb) layout src
11 (gdb) backtrace

```

دستور `layout` پنجره را به دو تکه تقسیم می‌کند که نشان می‌دهد gdb در کدام خط از برنامه قرار دارد. دستور `backtrace` تاریخچه توابع صدا زده شده در ریسمان فعلی را نشان می‌دهد.

پاسخ سوالات این قسمت را در قالب یک فایل pdf که دارای تصاویر انجام مراحل سوال است، تهیه نمایید و در سامانه‌های درس افزار شریف (CW) بارگذاری نمایید.

سوال ۱) با بررسی خروجی دستور `backtrace` بگویید کدام تابع `syscall` را صدا زده است.

چند بار `n` را فشار دهید تا از خط `; myproc() = *p` عبور کنید. پس از عبور از این دستور، عبارت زیر را اجرا کنید. این دستور `proc struct` پرده فعلی را به صورت مبنای ۱۶ چاپ می‌کند (فایل `kernel/proc.h` را مشاهده کنید).

```

1 p/x *p

```

سوال ۲) مقدار `p->trapframe->a7` را بدست آورید. این مقدار چه چیزی را نمایش می‌دهد؟ (راهنمایی: فایل `user/initcode.S` که اولین برنامه کاربر اجرا شده توسط xv6 است را بررسی کنید).

پرده در حال اجرا در حالت هسته می‌باشد و می‌توانیم ثبات‌های `privileged` مانند `sstatus` را چاپ کنیم (برای توضیحات بیشتری می‌توانید به این مستند مراجعه کنید):

```

1 (gdb) p/x $sstatus

```

سوال ۳) حالت قبلی‌ای که پردازنده در آن قرار داشت چه بود؟

ب- در این بخش از سوال می‌خواهیم که به سیستم‌عامل xv6 یک فراخوانی سیستمی اضافه کنیم که بتوان به کمک آن اطلاعاتی درباری وضعیت فعلی سیستم‌عامل و پرده‌های در حال اجرا به دست آورد. به طور کلی این فراخوانی سیستمی یک نمونه ساده‌تر از دستور `top` در لینوکس است.

خروجی این دستور باید شامل الف) PID پرده‌ها، ب) دستوری که به واسطه آن اجرا شده‌اند و ج) مقدار حافظه اختصاص داده شده باشد.

برای پیاده‌سازی این دستور نیاز است تا مشابه بقیه فراخوانی‌های سیستمی، کد آن را در بخش kernel سیستم عامل بنویسید و سپس یک برنامه سطح کاربر بنویسید تا آن را صدا بزند و خروجی نمایش داده شود. در نهایت از کارکرد فراخوانی سیستمی خود یک گزارش کوتاه تهیه کنید و آن را علاوه بر کدهای خود، در سامانه cw بارگذاری کنید. یک نمونه خروجی از اجرای این دستور را می‌توانید مشاهده کنید:

1	\$ top		
2	PID	Command	Size of Process Memory
3	1	init	16384
4	2	sh	20480
5	3	top	16384

در فیلم آموزشی موجود در cw توضیحات بیشتری درباره‌ی نحوه پیاده‌سازی این فراخوانی وجود دارد و حتما پیشنهاد می‌شود که آن را مشاهده کنید.

ج- اختیاری: فراخوانی سیستمی خود را به گونه‌ای پیاده‌سازی کنید که PID را به عنوان آرگومان ورودی مشابه ورودی زیر از کاربر بگیرد و سپس فقط داده‌های مرتبط با همان پردازش را خروجی دهد.

```
1 top -p <PID>
```

د- اختیاری: در فایل trap.c درست قبل از صدا کردن تابع syscall از تابع intr_on استفاده می‌شود که وقفه‌ها فعال شود. همچنین در اول تابع usertrapret وقفه‌ها غیر فعال می‌شوند. علت این موارد را توضیح دهید.

ه- اختیاری: در معماری AMD64 یا همان x86_64 دو روش کلی برای اجرای یک فراخوانی سیستمی وجود دارد: دستور **int** که یک وقفه^{۲۸} نرم‌افزاری ایجاد می‌کند و دستور **syscall** که یک فراخوانی سیستم است. این دو روش را با هم مقایسه کنید و توضیح دهید در هر کدام از لینوکس‌های نسخه ۳۲ بیتی و ۶۴ بیتی از کدام یک از این روش‌ها استفاده می‌شود.

۲. در این تمرین قصد داریم مقایسه‌ای کلی روی ساختار و کارایی محیط اجرای Bare Metal، ماشین مجازی و داکر داشته باشیم. برای انجام این تمرین لازم است که یک سیستم عامل لینوکس نصب شده بدون هیچ مجازی‌سازی به صورت نصب شده داشته باشید. همچنین داکر نیز باید روی سیستم عامل بدون مجازی‌سازی نصب شده باشد. روی هر سه محیط ابزار sysbench را نصب کنید.

توضیح تمرین: در این تمرین باید سه محک^{۲۹} زیر را در هر محیط اجرا و کارایی آن‌ها را مقایسه کنید:

محک cpu-intensive:

```
1 sysbench cpu --threads=4 --cpu-max-prime=2000000 --percentile=99 run
```

محک memeory-intensive:

```
1 sysbench memory --memory-total-size=20G --memory-access-mode=rnd run
```

محک io-intensive:

```
1 sysbench fileio --file-total-size=4G prepare
2 sysbench fileio --file-total-size=4G --file-test-mode=seqwr --percentile=99 run
```

سوال: عملکرد هر یک از این محک‌ها را به صورت جداگانه در حد یک خط توضیح دهید.

سوال: بر اساس درک خود پیش‌بینی از نحوه عملکرد هر کدام از محک‌ها در هر محیط ارائه دهید. شما لازم است که برای هر محک مقایسه‌ای از کارایی آن‌ها ارائه دهید و به صورت خلاصه علت پیش‌بینی خود را توضیح دهید.

حال هر یک از محک‌ها را روی هر سه محیط اجرا کنید و به ازای هر کدام یک نمودار از مقایسه موارد زیر بسازید:

²⁸Interrupt

²⁹Benchmark

● محک cpu-intensive:

- P99 Latency

- Average Latency

● محک memory-intensive:

- Total Time

- Throughput

● محک io-intensive:

- P99 Latency

- Average Latency

- Write Throughput

سوال: آیا نتایج بدست آمده با پیش‌بینی‌های شما یکسان بود؟ در صورت تفاوت با پیش‌بینی شما در هر مورد، در رابطه با نحوه اجرای برنامه در هر محیط تحقیق کنید و نتایج را توجیه کنید.

موارد لازم جهت تحویل: پاسخ به **سوال‌ها**، سه نمودار و توضیح کوتاه از مراحل کار.

۳. Fork Bomb

یکی از مواردی که می‌تواند سیستم را دچار مشکل کند ایجاد Fork به صورت انفجاری است. در این تمرین قصد داریم به بررسی مشکلاتی که Fork Bomb ایجاد می‌کند و نحوه رفع آن بپردازیم.

*** در این سوال توصیه می‌شود که از ماشین مجازی خود برای انجام آزمون‌ها استفاده کنید ***

۱. مفهوم Fork Bomb و نحوه عملکرد آن را توضیح دهید.

۲. یک نمونه Fork Bomb ساده بنویسید و آن را اجرا کنید.

۳. اثرات Fork Bomb روی منابع سیستم را تحلیل کنید. آیا مشکل با یک Ctrl+C ساده حل می‌شود؟ چرا؟

۴. راه‌های مقابله با Fork Bomb را شناسایی کنید.

۵. یک بار Fork Bomb را اجرا کنید، سپس نشان دهید که نمی‌توانید هیچ پردازنده دیگری بسازید، در نهایت Fork Bomb را مهار کنید.

نمایش و تحلیل کارایی برنامه

بهینه‌سازی عملکرد نرم‌افزار همواره به عنوان یکی از مراحل حیاتی در توسعه برنامه‌های کاربردی مطرح بوده است. در دنیای امروز که نیاز به پردازش داده‌های حجیم و اجرای محاسبات پیچیده روزبه‌روز در حال افزایش است، شناسایی گلوگاه‌های عملکردی و رفع آن‌ها نه تنها باعث بهبود سرعت اجرا می‌شود، بلکه مصرف منابع سیستمی را نیز بهینه می‌کند. در این راستا، ابزارهای تحلیلی مختلفی برای ارزیابی عملکرد برنامه‌ها توسعه یافته‌اند که هر یک با ارائه بینشی دقیق از نحوه اجرای کد، به برنامه‌نویسان در شناسایی بخش‌های بحرانی کمک می‌کنند.

برای این منظور، از ابزار **perf** در محیط لینوکس بهره می‌گیریم که امکان جمع‌آوری داده‌های عملکردی در سطح هسته سیستم عامل و سخت‌افزار را فراهم می‌سازد. علاوه بر این، با استفاده از **FlameGraph**، این داده‌ها را به صورت گرافیکی و قابل درک تجسم و تحلیل خواهیم کرد. در نهایت، با به کارگیری ابزار **Tracy**، که یک تحلیل‌گر عملکرد بلادرنگ است، جنبه‌های دیگری از رفتار برنامه را مورد بررسی قرار می‌دهیم. هدف نهایی این تمرین، نه تنها آشنایی با این ابزارها، بلکه درک عمیق‌تر از فرآیند بهینه‌سازی و تأثیر آن بر عملکرد کلی برنامه است.

۱. در مورد ابزار perf در لینوکس تحقیق کنید.
۲. یک کد زبان c/c++ پیچیده دارای تعدادی بخش نابهنه بنویسید که حداقل ۱۰ ثانیه اجرای آن طول بکشد (برای اینکار می‌توانید از ابزارهای هوش مصنوعی استفاده کنید)
۳. با استفاده از ابزار perf عملکرد آن را بررسی کنید.
۴. در مورد flamegraph تحقیق کنید.
۵. با استفاده از خروجی perf یک flamegraph بسازید.
۶. با استفاده از خروجی flamegraph بخش‌های نابهنه را پیدا کنید و آن‌ها را بهبود دهید.
۷. نتیجه بهینه‌سازی خود را درون flamegraph نمایش دهید.
۸. در مورد tracy تحقیق کنید.
۹. برنامه خود را به کمک tracy تحلیل کنید.